

Property 2: Non-Inclusive

Verifying that the DRAM cache and the on-chip LLC do not have to hold matching contents

1. What "Non-Inclusive" Means

In some cache designs, there is a strict rule between two levels of cache: an **inclusive** relationship means the bigger, lower-level cache must always hold a copy of everything the smaller, higher-level cache holds. If the lower level ever throws a line away, it must first tell the upper level to throw its copy away too — otherwise the upper level would be holding data the lower level no longer backs up.

A **non-inclusive** relationship means there is no such promise. The two caches are managed independently. Either one can hold a line the other doesn't have, either one can throw a line away without telling the other, and the system is still correct. Neither cache is a guaranteed subset or superset of the other.

Simple analogy: *Imagine two friends who each keep their own personal notebook copying down notes from the same lecture (the LLC is one notebook, the DRAM cache is the other). If one friend erases a page from their own notebook because they don't need it anymore, they don't have to run over and rip the same page out of the other friend's notebook. Each notebook is managed on its own. One notebook might have a page the other one threw away weeks ago — and that's completely fine, because neither notebook ever promised to always match the other.*

2. Why This Property Needs to Be True

The paper's design deliberately keeps the DRAM cache manager as an independent structure that receives traffic from "cores/LLC/DMA" and manages its own tag and metadata storage separately from whatever is happening in the on-chip caches above it. If the ChampSim port secretly forced the DRAM cache to always mirror the LLC's contents (or the other way around), it would need extra invalidation traffic flowing between the two levels — which would also break Property 1 (Out of Coherence), since that traffic would itself be a coherence

mechanism. Verifying non-inclusion is really checking that no such hidden synchronization exists anywhere in the code.

CLAIM TO VERIFY

When the LLC evicts a clean line, it does not tell the DRAM cache to remove anything. When the DRAM cache evicts a line during a miss, it does not tell the LLC to remove anything either. The two structures' contents are allowed to disagree at any point in time.

3. Proof — Evidence From the Actual Source Code

1 When the LLC evicts a CLEAN line, nothing at all is sent to the DRAM cache

This is the actual eviction code that runs inside the LLC when it needs to make room for a new line (a "fill"). Look closely at the condition that guards the only message sent downward:

src/cache.cc · lines 83–115

```
if (block[set][way].dirty) {           // only enters this block if DIRTY

    if (lower_level->get_occupancy(...) == lower_level->get_size(...)) {
        do_fill = 0;
        lower_level->increment_WQ_FULL(...);
    }
    else {
        PACKET writeback_packet;
        ...
        lower_level->add_wq(&writeback_packet); // tells the DCM about the eviction
    }
}
// if the victim was CLEAN, this entire block is skipped --
// do_fill simply stays 1 and the LLC overwrites its own slot silently.
```

The `lower_level->add_wq(...)` call — the only message the LLC ever sends to the DRAM cache about an eviction — sits strictly inside `if (block[set][way].dirty)`. If the line being replaced was clean, this whole block does not execute at all. The LLC just quietly reuses its own slot for the new line. The DRAM cache, sitting below it, is never told that line existed, let alone that it was thrown away.

2 When the DRAM cache evicts a line, it never notifies the LLC

Every single call the DRAM cache manager makes upward, toward the LLC, was searched for:

grep search across src/dram_cache_manager.cc for "upper_level"

```
upper_level_icache[i] = this;           // wiring setup only, line 98-101
upper_level_dcache[i] = this;

upper_level_icache[packet->cpu]->return_data(packet); // line 234
```

```
upper_level_dcache[packet->cpu]->return_data(packet); // line 236

upper_level_icache[resp.cpu]->return_data(&resp); // line 1143
upper_level_dcache[resp.cpu]->return_data(&resp); // line 1145

upper_level_icache[r.pkt.cpu]->return_data(&r.pkt); // line 1169
upper_level_dcache[r.pkt.cpu]->return_data(&r.pkt); // line 1171
```

Every single one of these calls is `return_data(...)` — and `return_data` only ever delivers the answer to a request the LLC itself already sent down. There is no function call anywhere in this file that pushes an unsolicited message up to the LLC saying "I just evicted address X, remove your copy." The DRAM cache's own eviction logic, `classifyAndInstall()` (`src/dram_cache_manager.cc`, described fully in Property 6's proof), simply overwrites its own slot — it never reaches upward to touch the LLC at all.

DIRECTION 1: LLC → DRAM CACHE

A clean LLC eviction is completely silent. Only a *dirty* eviction sends anything downward, and even then it's just a normal writeback request, not an instruction to invalidate.

DIRECTION 2: DRAM CACHE → LLC

The DRAM cache never initiates a message upward on its own. It only ever replies to requests the LLC already sent it.

4. Summary of Evidence

#	What was checked	Location	Result
1	LLC's behavior when evicting a clean line	src/cache.cc:83–115	Downward message only exists inside the "dirty" branch; clean evictions send nothing
2	Every upward call the DRAM cache makes toward the LLC	src/dram_cache_manager.cc (upper_level_* call sites)	All calls are <code>return_data()</code> answering an existing request; none push an eviction notice

CONCLUSION

Neither direction of communication carries an "erase your copy" message. A clean line can vanish from the LLC while still sitting in the DRAM cache, and a line can vanish from the DRAM cache while the LLC still believes it holds a valid copy of the address above it — and in both cases nothing in the code treats this as an error. This is exactly what non-inclusion means: the two caches' contents are independent, and correctness of the system does not depend on them matching. This is consistent with the paper's model, where the DRAM cache manager is a self-contained structure with its own tag and metadata storage, not a mirror of whatever the on-chip caches are holding.

2

DIRECTIONS
CHECKED

0

SYNC MESSAGES
FOUND

1

GUARDED CALL
(DIRTY-ONLY WB)

✓

PROPERTY
VERIFIED